

Manipulating HTML Elements Using JavaScript

By the end of this exercise, you will:

- ✓ Learn how to dynamically create and manipulate HTML elements using JavaScript.
 - ✓ Use **event listeners** to respond to user actions.
 - ✓ Work with **arrays and loops** to manage data.
-

Task: Build a Simple To-Do List App

You will create a webpage where users can **add tasks, mark them as completed, and remove them**.

Step 1: Create the HTML Structure

In an HTML file, write the basic structure of the webpage.

```
html
CopyEdit
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do List</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin: 50px;
    }
    input, button {
      padding: 10px;
      margin: 5px;
    }
    ul {
      list-style-type: none;
      padding: 0;
    }
    li {
      padding: 8px;
      margin: 5px;
      background-color: #f4f4f4;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
  </style>
</head>
<body>
  <div>
    <input type="text" value="Add new task" />
    <button value="Add" />
  </div>
  <ul>
    <li><input type="checkbox" /> Buy groceries</li>
    <li><input type="checkbox" /> Clean the house</li>
    <li><input type="checkbox" /> Call mom</li>
  </ul>
</body>
</html>
```

```

        }
        .completed {
            text-decoration: line-through;
            color: grey;
        }
    </style>
</head>
<body>

    <h2>My To-Do List</h2>
    <input type="text" id="taskInput" placeholder="Enter a task">
    <button onclick="addTask()">Add Task</button>

    <ul id="taskList"></ul>

    <script>
        function addTask() {
            let taskInput = document.getElementById("taskInput");
            let taskText = taskInput.value.trim();

            if (taskText === "") {
                alert("Please enter a task!");
                return;
            }

            let li = document.createElement("li");
            li.innerHTML = `
                <span onclick="toggleTask(this)">${taskText}</span>
                <button onclick="removeTask(this)">✖</button>
            `;

            document.getElementById("taskList").appendChild(li);
            taskInput.value = "";
        }

        function toggleTask(task) {
            task.classList.toggle("completed");
        }

        function removeTask(button) {
            button.parentElement.remove();
        }
    </script>

</body>
</html>

```

Step 2: Understanding the Code

HTML Explanation

- **Input Field:** Users type their task here.
- **Add Task Button:** Calls the `addTask()` function when clicked.
- **Unordered List (`<ul>`):** Holds all tasks dynamically.

CSS Explanation

- **Tasks are displayed as list items (`<li>`).**
- **Completed tasks get a strikethrough (`.completed` class).**
- **Each task has a delete button (✖) to remove it.**

JavaScript Explanation

- `addTask()`:
 - Creates a new `<li>` element with the task text and a delete button.
 - Adds the task to the list.
- `toggleTask(task)`:
 - Marks the task as completed by toggling the `.completed` class.
- `removeTask(button)`:
 - Removes the task when the ✖ button is clicked.



Expected Outcome

1. Users can **type a task** and click **"Add Task"** to add it to the list.
2. Clicking on a task **strikes it through** (marks as completed).
3. Clicking the ✖ button **removes** the task from the list.



Additional Tasks:

You can also **extend your project** with these features:

1. **Save tasks in `localStorage`** so they remain after refreshing the page.
2. **Add a "Clear All" button** to remove all tasks at once.
3. **Add a timestamp** showing when each task was added.



Things to ensure you check in your work

1. Correct HTML structure
2. JavaScript functionality
3. CSS styling
4. Code quality & best practices



Final Outcome

By completing this, you will gain **practical JavaScript experience** while building something **useful and interactive**.